AI-Powered Real-Time Metro Safety Monitoring System

A Detailed Project Blueprint (KTU B.Tech Major Project — AI & Data Science)

---

1. Problem Statement

Conventional CCTV systems in metro/public transport settings are **purely passive** — footage is reviewed only after an incident occurs, offering no means of preventing harm in real time. This project builds an **AI-integrated real-time monitoring system** that shifts metro surveillance from reactive to proactive, covering four complementary safety concerns:

1. **Fight/altercation detection** — identify physical fights as they happen, not after.
2. **Crowd density monitoring** — flag overcrowded compartments before it becomes a safety hazard.
3. **Abandoned object detection** — flag unattended bags/items (security concern, bomb-threat protocol).
4. **Platform edge/track intrusion detection** *(stretch goal)* — alert if someone crosses onto the track when no train is present.

All four modules share a common pipeline: **detection layer → rule-based event logic → alert/notification system → unified dashboard.**

---

2. Why This Genuinely Needs AI (Not Rules)

This is worth stating explicitly in your report, because it's your strongest justification:

- **Fight detection**: A fight can't be defined by a fixed rule ("2 people close together" also describes a hug or a crowded queue). It requires learning *motion patterns over time* — the way pose keypoints move erratically/aggressively — which only a trained temporal model can capture.
- **Crowd density**: Accurately estimating "how many people" in a dense, overlapping crowd from a single camera angle is a well-known hard computer vision problem — simple counting (e.g., contour counting) fails badly with occlusion; needs a trained density-estimation model.
- **Abandoned object & platform intrusion**: These two are more rule-based/geometric (see Section 5) — good to be honest about this in your report. Their "AI" component is the underlying object/person detector (a trained deep learning model), even though the event logic on top is simpler.

---

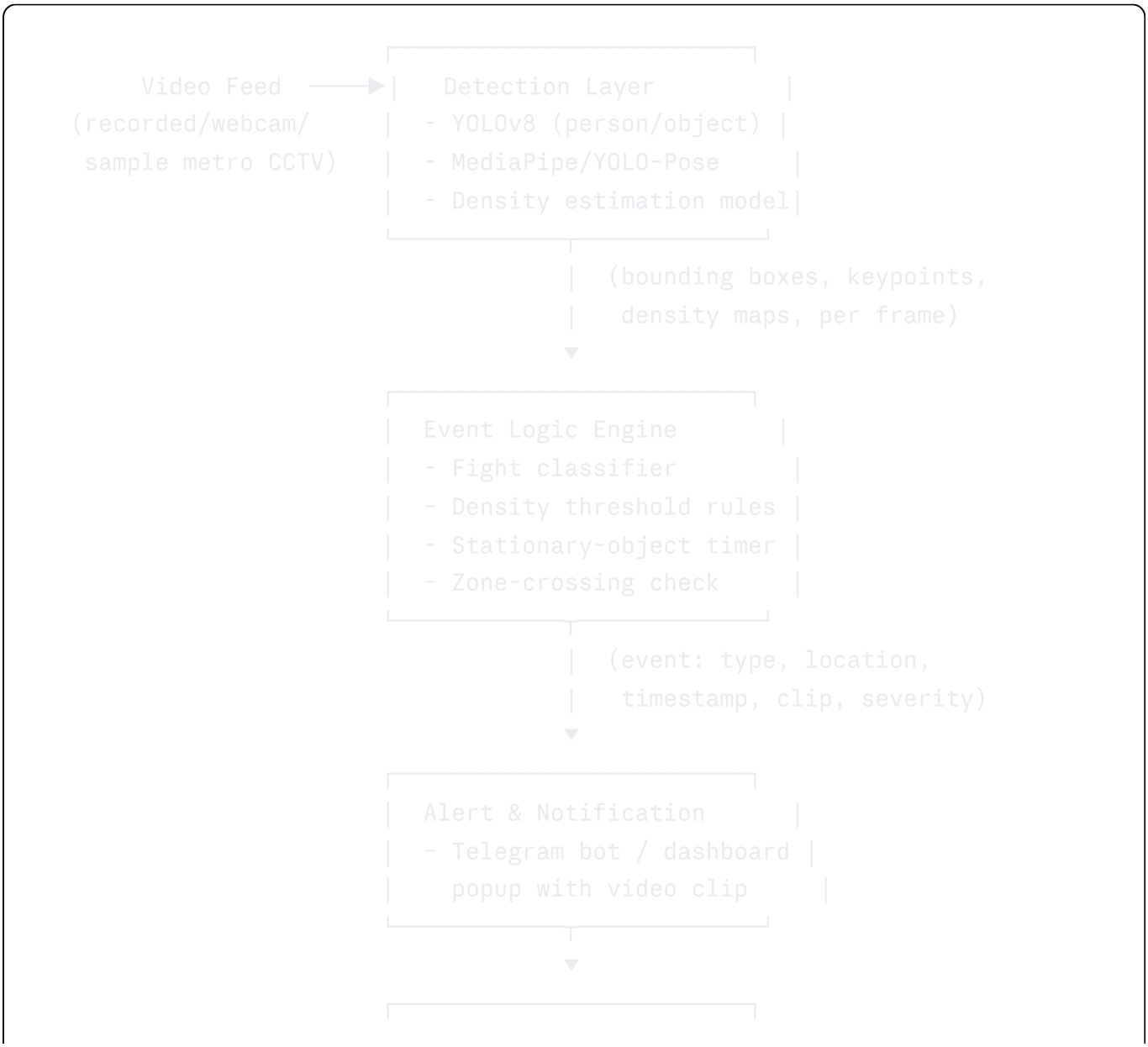
3. Recommended Module Priority (Given Your Scope Choice)

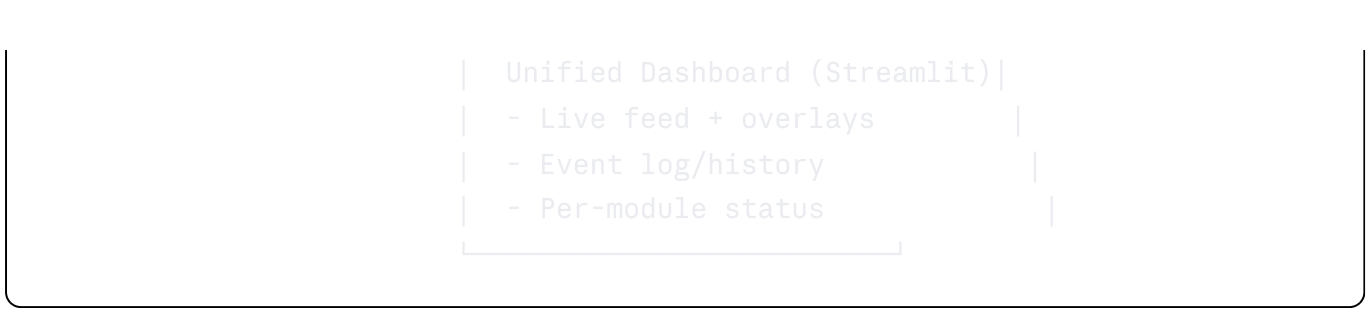
Priority	Module	Effort	"Wow" factor
1 (Core)	Fight/Altercation Detection	High	Very high
1 (Core)	Crowd Density Monitoring	Medium	High
2 (Core)	Abandoned Object Detection	Medium-Low	Medium-High
3 (Stretch)	Platform Edge/Track Intrusion	Low	High (if time allows)

Build in this order. By the time you finish the first two, you'll have a working pipeline (video → detection → alert) that the last two modules simply reuse with different logic — so they get progressively faster to build.

---

4. System Architecture





---

5. Detailed Methodology Per Module

Module 1 — Fight/Altercation Detection

Approach:

1. Use a **pretrained pose estimation model** (MediaPipe Pose or YOLOv8-Pose) to extract skeletal keypoints for each person, per frame — you are NOT training a pose model from scratch, just using it as a feature extractor.
2. From keypoints across a short time window (e.g., 15-30 frames ≈ 1-2 seconds), compute **motion features**: joint velocity, limb angle changes, proximity between two people's keypoints.
3. Train a **lightweight temporal classifier** (a small LSTM, or even simpler — a Random Forest/XGBoost on hand-engineered motion features) to classify a time-window as "Fight" vs "Normal activity."
4. **Dataset**: Look for public fight-detection datasets — e.g., **RWF-2000** (Real World Fights, 2000 video clips, widely used in academic fight-detection papers) or **Hockey Fight Detection Dataset**. Both are free/downloadable and specifically built for this exact task — you'll have direct literature and baselines to compare against.

Why this is achievable for a beginner: you're not designing pose estimation (pretrained handles that) — you're building a much smaller, simpler classifier on top of extracted features, which is very learnable even with your current DL comfort level.

Module 2 — Crowd Density Monitoring

Approach (pick based on time/comfort):

- **Simpler version (recommended to start)**: Use YOLOv8 (pretrained, `person` class) to **count** detected people per compartment/frame region, then classify density level (Low/Medium/High/Overcrowded) using simple thresholds calibrated against known compartment capacity. Good enough for a working demo and much easier to implement.
- **Advanced version (if time allows)**: Use a proper **crowd counting model** (CSRNet or MCNN, pretrained weights available) that handles occlusion/overlapping people far better than simple detection-counting, especially in dense crowds where people block each other.

Dataset for testing: ShanghaiTech Crowd Counting dataset (public, standard benchmark) to validate your density estimation approach, plus your own sample metro/train footage (search YouTube for public metro CCTV-style footage, or simulate with any crowded indoor video) for the actual demo.

Output: Per-compartment density score → if a compartment crosses a threshold, flag as overcrowded; simultaneously identify comparatively empty compartments → this comparison is what makes the "redirect passengers" recommendation meaningful.

Module 3 — Abandoned Object Detection

Approach:

1. Run YOLOv8 (pretrained, COCO classes include `backpack`, `suitcase`, `handbag`) on each frame to detect bag-like objects.
2. **Track** each detected object across frames (use a simple tracker like `ByteTrack` or `DeepSORT`, both have ready-to-use implementations that pair well with YOLOv8) to know if the *same* bag has stayed in roughly the same position.
3. **Logic**: if a tracked bag-object remains stationary (position doesn't change beyond a small tolerance) for longer than a threshold (e.g., 5 minutes) **and** no person is detected within a defined proximity radius of it for that duration → flag as "unattended object" event.
4. This module is mostly **engineering logic on top of a pretrained detector + tracker** — a great one to build once your pipeline from Module 1/2 already works, since it reuses the same YOLO detection layer.

Module 4 — Platform Edge / Track Intrusion (Stretch Goal)

Approach:

1. Manually define a **virtual boundary line/zone** near the platform edge in the camera frame (one-time setup per camera view — just pixel coordinates you specify).
2. Use the same person detector (YOLOv8) to get each person's foot position (bottom-center of their bounding box is a reasonable proxy).
3. **Logic**: if any detected person's position crosses the defined boundary **and** no train is present (you can simulate "train present" as a manual toggle for the demo, or detect it as a separate simple state) → trigger an instant alert.
4. This is the simplest module technically (geometry + an existing detector) — good to save for last since it's a fast, high-impact addition once your core pipeline exists.

---

6. Tech Stack Summary

Component	Tool
Language	Python
Object detection	YOLOv8 (Ultralytics — pretrained, easy API)
Pose estimation	MediaPipe Pose or YOLOv8-Pose
Object tracking	ByteTrack or DeepSORT
Fight classifier	scikit-learn (RF/XGBoost on motion features) or small LSTM (PyTorch)
Crowd density (advanced)	CSRNet (pretrained weights available on GitHub)
Dashboard	Streamlit
Alerts	Telegram Bot API
Video handling	OpenCV

7. Datasets Summary

Module	Dataset
Fight detection	RWF-2000, Hockey Fight Detection Dataset
Crowd density	ShanghaiTech Crowd Counting Dataset
Abandoned object / intrusion	No specific labeled dataset needed — pretrained YOLOv8 (COCO classes) + your own sample metro/CCTV-style test footage (YouTube public footage or simulate with any indoor video)

8. Timeline (14–15 Week Semester Plan)

Week	Task
1–2	Literature review (fight detection papers, RWF-2000 baseline results); set up YOLOv8 + MediaPipe environments

Week	Task
3–4	Build fight detection: extract pose keypoints, engineer motion features, train classifier on RWF-2000
5	Evaluate fight detector; tune window size/threshold
6–7	Build crowd density module (start with YOLO-counting version)
8	Evaluate density module against ShanghaiTech benchmark subset
9	Build abandoned object detection (YOLOv8 + tracker + timer logic)
10	Integrate all 3 core modules into one pipeline + event logic engine
11	Build Streamlit dashboard (live feed + overlays + event log)
12	Add Telegram alert integration
13	(If time allows) Build platform intrusion module
14	End-to-end testing on sample footage; write report
15	Buffer / rehearse viva demo

9. Evaluation Metrics

Module	Metrics
Fight detection	Accuracy, Precision/Recall, F1 (on RWF-2000 test split) — Recall matters most (missing a real fight is worse than a false alarm)
Crowd density	Mean Absolute Error (MAE) against ground-truth counts (ShanghaiTech)
Abandoned object	False positive rate (how often it flags a bag someone briefly set down vs. genuinely abandoned) — tune your time threshold to balance this
Platform intrusion	Detection latency (how fast it triggers alert after crossing)

10. Your Viva Demo Plan (This Sells the Project)

Prepare **3 short pre-recorded/simulated clips:**

1. A clip with a staged "fight" (even two people play-acting works for demo purposes) → show live detection + alert firing.
2. A crowded vs. empty compartment comparison clip → show density scores updating and the "redirect" recommendation.
3. A clip with a bag left behind → show the timer counting and the alert firing once the threshold is crossed.

Live, real-time demos showing the alert *actually firing* during your viva will make this project memorable — far more than a slide full of accuracy numbers.

---

11. Known Limitations (State These Proactively)

- Trained/tested primarily on public benchmark datasets and sample footage, not real deployed metro CCTV — a real deployment would need retraining/calibration to the specific camera angles and lighting of actual metro stations.
- Fight detection accuracy depends heavily on camera angle and occlusion — acknowledge this as a real-world deployment challenge.
- Abandoned object detection threshold (e.g., 5 minutes) is a tunable parameter with an inherent false-positive/false-negative trade-off — explain how you chose your threshold.
- Real-time performance (frames-per-second) will depend on hardware; mention what hardware you tested on (e.g., "achieves X FPS on Google Colab GPU / your laptop GPU").

---

12. Suggested Report/Viva Narrative

> *"Conventional CCTV systems function purely as passive recording tools, reviewed only after an incident has occurred. This project demonstrates that combining pretrained computer vision models (pose estimation, object detection) with purpose-built temporal and rule-based logic can shift public transport surveillance from reactive to proactive — detecting fights, monitoring crowd density, and flagging abandoned objects in real time, with alerts fired while an incident is still unfolding rather than discovered afterward."*

---

13. Extension Ideas (If You Finish Early)

- Add the platform intrusion module if not already done (Section 5, Module 4).
- Multi-camera support (aggregate alerts from multiple compartments/stations into one dashboard).
- Severity scoring — combine multiple simultaneous events (e.g., overcrowding + a developing fight) into an escalated priority alert.

- Historical analytics view — "which time of day/compartment has the most incidents" trend dashboard, useful for station planning.

---

14. Key References to Cite

- RWF-2000 dataset paper — *"RWF-2000: An Open Large Scale Video Database for Violence Detection"*
- ShanghaiTech Crowd Counting dataset paper (Zhang et al., CSRNet-related literature)
- Ultralytics YOLOv8 documentation
- Search terms for further literature review: *"video-based violence detection deep learning," "crowd density estimation CNN," "abandoned object detection surveillance"* — all well-established academic search terms with plenty of citable prior work.